

KV Cache 分層研究：文獻盤點、SOTA 的問題與下一步

日期：2026-09-30

範圍：延續 9/19 的回覆，整理這兩週讀過的約 60 篇論文與系統、兩個平台上的量測，以及我接下來打算怎麼做。

標註方式

標註	意思
〔實測 A〕	平台 A (7 × RTX 3090) 量到的，出處是 results/RUNLOG.md
〔實測 B〕	平台 B (1 × MI300X) 量到的，出處是 results/RUNLOG_MI300X.md、EXPERIMENTS_20260919.md
〔初步〕	已經跑過，但腳本還沒依記錄協定入庫，重跑前不當定論
〔文獻 n〕	論文或系統原文，編號對應文末參考資料
〔推論〕	我的判讀，還沒有實驗支持
〔算術〕	用實測常數計算出來的，不是量測

1. 現況：這一塊做的都是 KV cache 優化

我讀完這批論文後的第一個感覺是：大家做的都是「KV cache 優化」，但切入的層次差很多，所以彼此很難比較。我把它們整理成六層，每一層回答一個問題。

先用一個比喻：KV cache 可以想成模型讀過一段文字後留下的筆記。下次遇到同一段文字就直接翻筆記，不用重讀（重讀就是重算，也就是 prefill）。

L5 誰在用、用多久？	← 使用方式與契約
L4 怎麼找到筆記？	← 命中規則
L3 怎麼搬筆記？	← 傳輸
L2 筆記放哪裡？	← 放置（原本的 Tiara 在這層）
L1 筆記寫多細？	← 精度／格式
L0 要存的是什麼筆記？	← 物件種類

L0：要存的是什麼

- 在做什麼：決定快取的單位是什麼。一般 Transformer 只要存每個 token 的 K、V，所以幾乎所有系統（vLLM、SGLang、LMCache、原本的 Tiara）都只管 KV。
- 正在改變的地方：Qwen3-Next、Qwen3.5、Kimi-Linear 這類 hybrid 模型，有一部分層是 linear attention。這些層存的不是每個 token 的 KV，而是一份會被就地覆寫的 recurrent state。
- state 每份約數十 MB，而且只在固定位置（例如每 4,096 token）存下來才有用〔文獻 20〕。
- SGLang 有一個重現：state 的空間用滿時，host 端的 KV 命中變成 0/8，當時 KV 池還空著 35%〔文獻 21〕。KV 還在，但缺了 state 就接不上。
- 另一個例子：同一個底座模型掛不同 LoRA 時，可以借用別人的 KV。代價是準確率最多掉約 5 分，而且只在 1–1.5K token 的短 context 測過〔文獻 22〕。

L1：筆記寫多細（精度）

- 在做什麼：用 BF16、FP8、INT8、INT4 存 KV，省空間但可能損失品質。
- 代表工作：KIVI、KVQuant、KVTuner 做量化；AdaptCache、EvicPress 主張壓縮和放置要一起決定；VeriCache、QuantSpec 用低精度當草稿、高精度驗證〔文獻 9、10、17、18〕。

L2：筆記放哪裡（放置）

- 在做什麼：決定每一段 KV 要放在 GPU、CPU DRAM、SSD，還是直接丟掉、要用時重算。
- 代表工作：Strata／SGLang HiCache、Cake、Pensieve、Bidaw、MTDS、LMCache、Mooncake，以及工業界的 vLLM OffloadingConnector、Dynamo KVBM〔文獻 1–8〕。
- 原本的 Tiara 就在這一層（見第 2 節）。

L3：怎麼搬筆記（傳輸）

- 在做什麼：把 KV 從 CPU／SSD 搬回 GPU，或在機器之間傳送。
- 大家的假設：傳輸時間 $\approx$ 資料量 $\div$ 頻寬。
- 我們量到的：搬運速度主要由軟體路徑決定（第 6.2 節）。

L4：怎麼找到筆記（命中規則）

- 在做什麼：決定什麼情況算「命中」。
- 現況：vLLM 與 SGLang 只認「從第一個 token 開始完全相同」的前綴，中間有一個 token 不同，後面就全部作廢。
- 往前推的工作：CacheBlend、EPIC、MEPIC 讓中段內容也能重用，但有品質損失〔文獻 12、23〕。

L5：誰在用、用多久（使用方式與契約）

- 在做什麼：處理負載和服務規則本身。
- 例子：
 - agent 多輪互動；
 - reasoning 模型產生很長的思考內容；
 - 商業 API 的快取其實是「保證存 5 分鐘」的契約，而不是單純的 LRU〔文獻 26、27〕。

這樣分之後看得比較清楚：L2 的論文最多也最擠；L0、L1、L3、L4 的變化最快；L5 則幾乎都被當成固定前提。

2. 原本的 Tiara 在做什麼，現在到哪裡

2.1 原本的主張（L2）

Tiara (Tiered, quality-Aware, Recompute-Aware) 把 KV 放置寫成「品質約束下、動作成本不同的線上決策」。

- 動作空間：GPU-BF16、GPU-FP8、GPU-INT4、CPU、SSD、DROP + 重算，共六態。
- 核心論點：重算與傳輸的成本比 κ 在不同硬體上差很多，所以不該所有硬體用同一套放置策略。
- 做法：用一個學習式預測器決定每個 block 的去向。

2.2 這兩週量到、會改變原本主張的事

原本的說法	現在的證據	狀態
κ 跨硬體差 32 倍	同一套 harness、Llama-8B、context 16K：κ_cpu 在 3090 是 8.85–9.53，在 MI300X 是 1.53，只差 5.8–6.2 倍。同一張 MI300X 換模型或 context，就從 1.53 變到 11.65〔實測 A、B〕	32 倍要撤掉，改報實測值
傳輸成本 ≈ 硬體頻寬	MI300X 的 CPU 階只用到 PCIe 的 4–8%；3090 的 SSD 階只有同一顆 NVMe 裸讀的約 1/18〔實測 A、B〕	推翻（第 6.2 節）
聰明的放置有很大空間	全 BF16 時，oracle 比最好的 baseline 少 20.66%/20.91% 的 prefill 時間；只把 KV 改成 INT4、配 vLLM 原廠 LRU，就拿到這個空間的 91.8%/103.9%，剩下 2.02%/3.52%〔實測 B，初步〕	在延遲層面基本推翻（第 6.1 節）
學習式預測器是主要貢獻	3090 合成長上下文：拿到 oracle 空間的 81.85%〔實測 A〕。MI300X：輸給最佳 baseline (tier_fs) 9.7–47.3%〔實測 B〕	只在階層差距大時成立
Tiara 掛在 vLLM 的 CachePolicy 介面上	這個介面只能決定「CPU 階丟誰」，六態裡有四態表達不了；Tiara 的策略目前只在模擬器裡跑過，沒有真機端到端〔OPEN_ISSUES.md B7〕	論文要改寫

我的結論：「在 L2 做一個更聰明的放置策略」這條路已經很擠，而且我們自己的量測也顯示它剩下的空間不大。但這些量測本身（傳輸看軟體、INT4 取代放置）是別人沒量過、而且會推翻現有假設的現象，這部分值得留下來。

3. SOTA 整理：每篇在解什麼、強在哪、哪裡確實做不到

「確實做不到」一欄只寫原文或我們核對過的內容。

論文／系統	層	在解什麼	最強在哪	確實做不到的地方
Strata (OSDI'26) / SGLang HiCache〔文獻 1〕	L2 + L3	GPU→CPU→儲存三層；用 GPU 協助搬運，並改記憶體佈局	已經在 SGLang 生產環境使用；傳輸做得很快	逐出只用 LRU（原文：「For all memory layers, the LRU algorithm serves as the default eviction policy」）；沒有量化；SSD 只出現在一個子實驗；認為長 context 的重算「an unattractive alternative」
Cake (ICML'25)〔文獻 2〕	L2	前段重算、後段從儲存載入，兩邊並行	同時用上算力和 I/O	沒有成本模型，也沒有 fallback（作者列為未來工作）；I/O 是用算的延遲模擬，沒有真實儲存裝置；假設 KV 都已存好；Table 6 有設定比單純載入還慢 (0.80x)
Pensieve (EuroSys'25)〔文獻 3〕	L2	依「重算成本 ÷ 閒置時間」逐出，先丟前段	最早把「位置越後面重算越貴」用在逐出上	只有 GPU/CPU，沒有 SSD，也沒有精度
Bidaw (FAST'26)〔文獻 4〕	L2	多輪對話的分層快取	有 Belady 命中率上界；trace 公開；在 vLLM 上重做開源對手	上界是命中率，不是延遲；「多存一份 tensor 換計算」只對 MHA 划算

論文／系統	層	在解什麼	最強在哪	確實做不到的地方
MTDS (C&IS'26) [文獻 5]	L2	每個請求決定全載、部分載或不載	量到載入時間 「non-linear slow-then-fast」，命中很長時不載反而快	結論與 Strata 相反；4 × A10、ShareGPT 加 Poisson 到達
AdaptCache (BigMem'25) [文獻 9]	L1 + L2	依品質曲線決定每個條目的壓縮率與放置	最早主張壓縮和放置要一起做	用歷史頻率預測未來 (LFU 式)；品質曲線每個資料集只抽 10 筆、用 GPT-4o 生問題；沒有 oracle；5 頁 workshop
EvicPress (2025) [文獻 10]	L1 + L2	逐出與壓縮聯合決定	聯合比單做好 2.19×	沒有 oracle 上界，不知道離最佳解多遠
LMCache (2025) [文獻 6]	L2 + L3	vLLM / SGLang 的 CPU / 磁碟卸載外掛	生產級，被很多系統整合	最大倍數 (如 14×) 出現在 baseline 已接近飽和的工作點
Mooncake (FAST'25) [文獻 7]	L2 + L3	以 KV 為中心的分離式架構 (Kimi)	生產級；trace 公開	需要多節點與 RDMA
vLLM OffloadingConnector / Dynamo KVBM [文獻 8]	L2	生產環境的分層規則：LRU / ARC、store_threshold、用過兩次才寫 SSD	規則簡單，已在生產使用	規則固定，不看硬體也不看位置
py-kvcache (2026-09) [文獻 11]	L2	依硬體量出損益平衡點，決定存不存、載不載	會依硬體調整	以整段前綴為單位；沒有精度
CacheBlend (EuroSys'25) / MEPIC [文獻 12、23]	L4	讓中段內容也能重用	解決「只認開頭」的限制	有損；每個請求各自修補的 KV 在請求之間分歧，不能共享頁
Marconi (MLSys'25) / DASC (2026-08) [文獻 19、20]	L0	hybrid 模型 state 的保留與壓縮	最早處理 hybrid 模型	只在 GPU 內處理；DASC 全文搜 offload、CPU、SSD、DRAM 都是 0 次
VeriCache (2026-05) / QuantSpec (ICML'25) [文獻 17、18]	L1 + L2	低精度 KV 當草稿，完整 KV 驗證	greedy 下輸出與完整 KV 相同	VeriCache 同一份資料存兩份，沒有 SSD；QuantSpec 全在 GPU 內

讀下來的判斷：沒有一篇同時決定「放哪一層、用幾個 bit、要不要重算」三件事〔文獻 1–10〕。但依我們的量測，同時決定三件事的價值也有限（第 6.1 節），所以我不打算用「只有 Tiara 三格一起做」當主要賣點。

4. 大家怎麼做評測：baseline 與資料集

9/24 統計了 35 篇論文的評測設定〔docs/research_20260924/workloads_eval.md〕：

- Baseline 幾乎不重疊：
- 有做比較的 32 篇，總共用了 64 種不同的 baseline，其中 41 種只出現在一篇。
- 任取兩篇，72.6% 完全沒有共同對手。

- 最常見的共同對手只有 LRU 和原版 vLLM（各 9 篇），兩個都是地板。
- ARC 在 35 篇裡 0 次被當 baseline。我們目前只跟 LRU／ARC 比，是因為 vLLM OffloadingConnector 內建這兩個，不是文獻慣例。
- 到達過程多半是假的：用真實時間戳的只有 2 篇（Bidaw、Mooncake）；用 Poisson 的 10 篇；完全沒有到達過程的 19 篇。
- 沒有人報誤差：重點五篇都沒有報 seed、變異數或信賴區間。
- 命中率沒有統一定義：同一份 Mooncake toolagent trace，依定義不同，命中率可以是 1.1%（整個 prompt 都命中）到 100%（至少命中一個 block）。有寫出公式的論文都用「token 加權、算到第一個缺口」。
- 資料集偏短：
 - AdaptCache 用的 LongBench 子集平均只有 1K–11K 字，而且我們依公開資料重算的筆數對不上它寫的 1,100 段。
 - MEPIC 的請求平均 1.4–2.2K token〔文獻 23〕。
 - LRAgent 的 context 是 1.0–1.5K token〔文獻 22〕。

我的做法：

- baseline 分四類：
- LRU／ARC 當地板；
- 生產系統的規則（vLLM `store_threshold`、SGLang HiCache、KVBM 式寫入過濾）；
- 論文的決策規則（Pensieve、Cake、py-kvcache），沒開源的就在同一底座上重做，Bidaw 就是這樣做的；
- 簡單 heuristic「全域 INT4 + LRU」。

另外加 oracle 當上界。 - 比較方式：固定比同一個 baseline，數字附信賴區間。因為「最佳 baseline 是誰」在成本常數的抽樣中有 27% 的機率會翻轉〔實測 B，初步〕。

5. 硬體：能做什麼、不能做什麼

5.1 設備

項目	平台 A	平台 B
GPU	7 × RTX 3090 24 GB (sm_86)	1 × AMD MI300X 192 GiB (容器只分到 1 張；主機有 8 張)
CPU／RAM	共用機器	32 核、434.8 GiB 配額； <code>/dev/shm</code> 178.8 GiB，也就是 CPU 階的上限
慢速階	本機 SSD	<code>/var/tmp</code> ：本地 overlay，底層是 Broadcom MegaRAID（不是 NVMe），讀取約 5,392 MiB/s，屬暫態儲存
軟體	vLLM v0.28.0 (CUDA)	vLLM v0.28.0 (ROCm)，跟平台 A 同版
限制	共用機器，要防止其他人搶用 GPU；消費卡沒有分軌能耗計數器	容器裡看不到 NVMe 與 RDMA；只能用單卡

5.2 Baseline 在 MI300X 上能不能重現

Baseline	狀態	說明
full_gpu、cpu_lru、cpu_arc、tier_fs (vLLM 原廠)	已跑	4 個模型：Llama-3.1-8B、Qwen2.5-7B-1M、Qwen3-30B-A3B、Seed-OSS-36B
全域 INT4 + LRU	已跑，初步	腳本入庫後重跑
Oracle	模擬器	用貪婪解，所以得到的空間是下界
vLLM store_threshold (近似 KVBM)	可以跑，還沒跑	改設定就行
Pensieve、Cake、py-kvcache 的決策規則	只能在模擬器重做	原版沒開源；要在真機上跑需要改 vLLM
SGLang HiCache (Strata 的上游實作)	可以試，還沒測	ROCm 修正 9/19、9/21 才合併
LMCache	可以試，很可能跑不起來	相依套件是 CUDA 專屬
Mooncake Transfer Engine、NIXL P/D	不能	需要多節點與 RDMA
Tutti (GPU 直讀 SSD)	不能	容器裡沒有 NVMe
能耗相關	不能	amd-smi 只有整卡功耗

結論：在 MI300X 真機上，能比的是 vLLM 原廠規則、INT4，以及 SGLang HiCache（如果能跑起來）。論文方法則在同一個模擬器上、用實測常數重做。

6. 我的觀察

6.1 L1 × L2：量化和放置在延遲上是替代關係

- 量到的：全域 INT4 加原廠 LRU，就拿到最佳放置 92–104% 的好處〔實測 B，初步〕。
- 原因：放置策略的價值，來自「GPU 放不下常用的 KV，只好挑」。INT4 讓每份 KV 小 3.77 倍，GPU 的等效容量也跟著放大 3.77 倍。我們 trace 上常用的 KV 剛好落在「原容量」和「3.77 倍容量」之間，所以改完之後幾乎都放得下，放置就沒事做了。
- 成立條件，至少要一起寫出來：
- 模型要撐得住 INT4。Qwen2.5 族在 FP8／INT4 下直接崩潰；Llama-8B、UltraLong-8B、Qwen3-30B-A3B、Seed-OSS-36B 在 129K 的撈針測試中五種精度都是 100%〔實測 A、B〕。
- 壓力不能太大。壓力大到「縮 3.77 倍也放不下」時，放置可能又有用。這個轉折點還沒量。
- 要全域降，不能逐 block 降。3090 上只降最冷的 block，降了 780,108 次，總時間只變 0.004%〔實測 A〕。
- 目前只看 prefill。在 paged 框架裡，4-bit KV 的 decode 吞吐是 FP16 的 0.88–1.02×〔文獻 24〕，所以端到端要另外量。

6.2 L3：搬運速度是軟體決定的，而且「命中率高」和「搬得快」會互相打架

- MI300X：
- vLLM 的 CPU 階在同一條 PCIe 上，換模型就差 17.1 倍 (2.27–38.28 GB/s)。
- microbenchmark 顯示每一筆傳輸描述符有約 13 μ s 的固定成本，跟資料大小無關〔實測 B，初步〕。

- 根因在程式碼：vLLM 在 ROCm 上停用 Triton 快速路徑，改走 `hipMemcpyBatchAsync`，而且受 ROCm 7.2.1 的 bug 限制。
- 3090：主設定的 SSD 階約只有同一顆 NVMe 裸讀的 1/18。如果改用裸讀頻寬計算，SSD 與重算的交叉點 P* 會從 37,717 token 變成 0〔實測 A；算術〕。
- 外部佐證：vLLM 在 NVIDIA GB200 的 P/D 路徑上也有人量到同樣的現象。
- 每次傳輸約 4.9 GB 要發 91k–120k 個描述符，每個描述符的成本依傳輸方式差 20 倍（`cuda_ipc` 4.067 μ s、`InfiniBand` 0.204 μ s）〔文獻 13〕。
- 關鍵觀察：prompt 從 10.3k 變成 41.1k token，描述符數量不變。描述符會變多，是因為 prefix 重用讓 block 散在記憶體各處，不是因為 context 變長。
- 推論：命中率越高，實體佈局越碎，搬得越慢。目前沒有論文把這一項放進成本模型〔推論，待量〕。
- 必須正面引用的前作：vLLM 官方 1 月已經寫過「把各層的小 block 合成一大塊，offload 吞吐提升約一個數量級」〔文獻 14〕。但我們在 MI300X（同版 v0.28）上，Qwen-7B 與 Llama-8B 還是每層一小塊，只有 Qwen3-30B-A3B 合併成 1.5 MiB。要先查清楚為什麼：
- 如果是 ROCm 特有的，這條就是「CUDA 上已修、ROCm 上仍在，量級 17 倍」；
- 如果只是設定問題，貢獻就要縮小。

6.3 文獻的結論互相矛盾，矛盾來自傳輸實作

- Strata 說長 context 重算不划算〔文獻 1〕；MTDS 量到命中很長時不載入反而快〔文獻 5〕。
- 我把 18 篇論文的評測平台放在同一條 κ 軸上〔`docs/research_20260924/kappa_map.md`〕：
- 11 篇的結論跟它所在的硬體區間一致；
- 3 篇要再加一個因子才解釋得通（軟體路徑、容量、GPU 忙碌度）；
- 另外 4 篇的動作不同，無法比較。
- 推論：兩篇的矛盾很可能只是各自站在傳輸速度軸的不同位置，而不是演算法有優劣〔推論〕。

6.4 對這批論文的整體看法

共同的優點

1. 都從「算」和「存」的取捨下手。一類處理 GPU/CPU/SSD 三層放置；另一類專攻 SSD→GPU 的讀回路徑。收益大多落在 prefill，也就是 TTFT。
2. 手法主要有三種：
3. I/O 感知加排程（OS 的經典技巧）；
4. 針對特定場景的人工規則；
5. 用模型預測。搬運通常可以做成非同步，和計算重疊。
6. 大多選多使用者、KV 重複度高的場景。收益集中在「長 context、短輸出、高重用」，例如 RAG、文件 QA、agent 共用的 context。
7. 個別機制其實都是簡單的 heuristic，例如「載入時間和重算時間比一比」「整組一起命中」「用 decode 填空檔」。它們的價值在框架：排程器必須同時看算力和搬運量。
8. 跟 vLLM 預設的「命中就載入」相比，常見的改進是多做一步判斷：先比較載入時間和重算時間，再決定全載、載一部分或不載；依 prefix 被重用的機率排優先權，低價值的延後處理；逐出從 LRU 改成考慮頻率。
9. Bidaw 的 trade-off 很好參考：多存一份中間 tensor 換掉部分計算，到底值不值得。它的結論是只對 MHA 划算。
10. 問題都講得很具體，會說明在什麼規模、什麼負載下成立。

11. Strata 用 Little's Law 說明「要把每次搬運的量放大」，這個說故事的方式值得參考。我們的描述符固定成本可以用同一條式子解釋；另外也要考慮 delayed hit，也就是同一段 KV 正在載入時又被請求的情況。

共同的短板

1. 資料集侷限：很多用合成資料，或依假設產生負載；到達過程多半是 Poisson，甚至沒有；多數 context 偏短。
2. 層次差很多，難以比較：baseline、資料集、命中率定義都不一樣（第 4 節）。
3. 大多只做一層，其他層用預設值，而且都預設「傳輸 $\approx$ 頻寬」「量化與放置互補」這類系統前提。
4. token 粒度要講精確：
5. 在單一請求內挑 token 讀的工作很多（Quest、ShadowKV、InfiniGen）。
6. 但做放置的系統多以整段 context 或 chunk 為單位，例如 EvicPress 明寫「context-level, instead of chunk-level ... or token-level」。
7. Pensieve、AsymCache、HCache 已經在位置或層的維度做了一部分。
8. 品質評估偏弱：
9. 不是沒人考慮品質：AdaptCache 用品質曲線決定壓縮率，KVtuner 做逐層精度。
10. 問題是樣本少（每集 10 筆）、只看平均分數。
11. 我們量到能不能降精度取決於模型和任務（Qwen2.5 一降就崩），平均分數會掩蓋這件事〔實測 A、B〕。
12. MLSys'25 的 Rethinking KV Compression 也指出平均分數下隱藏了大量個別樣本失敗〔文獻 24〕。
13. 沒考慮 reasoning 的 thinking 模式：
14. 模型先產生一大段思考再回答，client 下一輪照官方建議把思考刪掉。從思考開始，後面的 KV（連答案也算）在前綴規則下就永遠不會再命中〔文獻 15〕。
15. vLLM v0.28.0 的 OffloadingConnector 預設還會把這些 decode 產生的 block 卸載到 CPU/SSD（只有開 offload_prompt_only 才不會）。這是我讀原始碼確認的，還沒實際量。
16. SGLang 有一個預設關閉的開關，直接不存思考內容〔文獻 16〕。
17. hybrid 模型還沒被分層系統處理：
18. MLPerf v6.1 Agentic 已經採用 hybrid 模型，但現有的分層論文都假設只有 KV（第 1 節 L0）。
19. 這條線最近四週出現了四個相關工作，競爭會很快。

6.5 「最強方法在什麼 workload、什麼硬體條件下會做錯決定」

先把每個 SOTA 的決策前提寫出來，再用我們量到的事實找出前提失效的地方。下表右欄是預測，還要實驗確認。

SOTA 的前提	在哪裡失效（我們的證據）	預期會做錯的決定
傳輸時間 = 資料量 ÷ 硬體頻寬（Cake、多數論文）	MI300X 上同一條 PCIe，換模型就差 17 倍	載入／重算的切點算錯
長 context 載回一定比重算划算（Strata）	3090 的 SSD 路徑只有裸讀的 1/18，交叉點隨實作移動	中短請求應該重算卻去載入
模型撐得住低精度（全域 INT4）	Qwen2.5 在 FP8/INT4 下崩潰	品質崩掉
用過的次數能預測未來（KVBM、HiCache 的寫入過濾）	reasoning 的思考 block 寫入後永遠不會命中	把不會再用到的 block 寫進 SSD

SOTA 的前提	在哪裡失效（我們的證據）	預期會做錯的決定
只有 KV 需要管理	hybrid 模型的 state 池滿時，KV 命中歸零	以為 KV 還在，實際上接不上

做法：在真實 trace（Mooncake 全量、TraceLab）與兩個平台上，逐次記錄每個方法跟 oracle 不同的決策，量出「錯誤決策數」和「比 oracle 多花的時間」，最後畫成一張地圖（橫軸是傳輸成本，縱軸是壓力或模型能不能降精度），標出每個方法在哪些區域會做錯。

Tiara 的角色也要改：不再是「更聰明的預測器」，而是「先量出這台機器、這條軟體路徑、這個模型的幾個常數，再決定用 INT4、用放置策略，還是改成重算」。它不犯同樣錯誤的理由，是這些常數是量出來的，不是寫死的假設。如果地圖上有 Tiara 也做錯的地方，我會照實報。

7. 接下來要做什麼

時間	工作	目的
本週	查清楚 vLLM v0.28 在 ROCm 上為什麼沒有用跨層合併的 block 佈局；同時確認 3090 上實際的佈局	決定第 6.2 節的定位
本週	9/19 三個實驗（INT4 取代放置、描述符固定成本、成本常數不確定性）依記錄協定重跑，腳本進 code/、CSV 進 results/	目前這些數字還不能引用
本週	補上 MI300X 的摘要 CSV（目前 GitHub 上一個都沒有）	讓每個數字都能追溯
第 2 週	3090 上量 reasoning 模型「寫進 CPU／SSD 卻永遠不會被讀回」的比例	驗證第 6.4 節短板 6，成本低
第 2 週	MI300X 的 connector 計數器加上「描述符數 vs 命中率」	驗證「命中越高、搬得越慢」
第 2 週	壓力掃描，找出量化從「替代」變成「互補」的轉折點	決定第 6.1 節能不能寫成通則
第 3 週	用小型 hybrid 模型（Qwen3.5-4B）在 3090 上試跑一週：state 會不會被卸載、會不會命中歸零、INT4 的結論還成不成立	決定 L0 要不要成為下一篇的主題
第 3 週	做第 6.5 節的錯誤決策地圖；試跑 SGLang HiCache 在 MI300X 上能不能用	回答老師的問題
之後	品質：RULER、LongBench、撈針，每格 $n \geq 500$ ，附信賴區間	讓「能不能降精度」有統計檢定力

需要老師決定的：

- 第一篇能不能是分析型？也就是「決定 KV 分層效益的是量化、傳輸實作和 prefill 佔比，不是放置策略」，加上一個事前判準。
- 如果一定要新機制，候選有兩個：
- 把 INT8 拆成兩個 4-bit 平面，前半留 GPU、後半放 CPU／SSD（L1 + L2）；
- hybrid 模型的 state 與 KV 聯合分層（L0）。
- 投稿：MLSys 2027（10/30）時程很緊；ICPE 或 EuroMLSys 比較穩。

參考資料

1. Xie et al., Strata, OSDI'26 ; SGLang HiCache 設計文件 :
https://docs.sglang.io/advanced_features/hicache_design.html
2. Jin et al., Cake, ICML'25 : <https://arxiv.org/abs/2410.03065>
3. Pensieve, EuroSys'25
4. Hu et al., Bidaw, FAST'26 : <https://www.usenix.org/system/files/fast26-hu-shipeng.pdf> ; trace :
<https://github.com/ShipengHu-777/Interactive-conversation-workload>
5. MTDS, Complex & Intelligent Systems, 2026 :
<https://link.springer.com/content/pdf/10.1007/s40747-025-02200-4.pdf>
6. LMCACHE : <https://arxiv.org/abs/2510.09665>
7. Mooncake : <https://arxiv.org/abs/2407.00079>
8. vLLM KV offloading 文件 : https://docs.vllm.ai/en/latest/features/kv_offloading_usage/ ; Dynamo KVBM : <https://docs.nvidia.com/dynamo/>
9. AdaptCache : <https://arxiv.org/abs/2509.00105>
10. EvicPress : <https://arxiv.org/abs/2512.14946>
11. py-kvcache : <https://arxiv.org/abs/2609.11744>
12. CacheBlend, EuroSys'25 : <https://arxiv.org/abs/2405.16444>
13. vLLM issue #55434 (GB200 NIXL 描述符) : <https://github.com/vllm-project/vllm/issues/55434>
14. vLLM blog, KV Offloading Connector (2026-01-08) : <https://vllm.ai/blog/2026-01-08-kv-offloading-connector>
15. vLLM issue #39321 (thinking token 污染 prefix cache) : <https://github.com/vllm-project/vllm/issues/39321>
16. SGLang PR #23315 (--strip-thinking-cache) : <https://github.com/sgl-project/sglang/pull/23315>
17. VeriCache : <https://arxiv.org/abs/2605.17613>
18. QuantSpec, ICML'25 : <https://arxiv.org/abs/2502.10424>
19. Marconi, MLSys'25 : <https://arxiv.org/abs/2411.19379>
20. DASC : <https://arxiv.org/abs/2608.30386>
21. SGLang PR #39436 (hybrid 模型 host 階的 state 池) : <https://github.com/sgl-project/sglang/pull/39436>
22. LRAgent, ICML'26 : <https://arxiv.org/abs/2602.01053>
23. MEPIC : <https://arxiv.org/abs/2512.16822>
24. Rethinking KV Cache Compression, MLSys'25 : <https://arxiv.org/abs/2503.24000>
25. Bottlenecks, MLSys'26 : <https://arxiv.org/abs/2601.19910>
26. Anthropic prompt caching 文件 : <https://platform.claude.com/docs/en/build-with-claude/prompt-caching>
27. Khailo, Keepalive Economics for Agentic Workloads : <https://arxiv.org/abs/2607.19214>

詳細證據與逐條查證紀錄 : [docs/DIRECTIONS_20260924.md](#)、[docs/DIRECTIONS_20260927.md](#)、[docs/research_20260924/](#)、[docs/research_20260927/](#)。